

```
1 import java.util.concurrent.locks.Lock;
2 import java.util.concurrent.locks.ReentrantLock;
3
4 public class DeadLock {
5     private Lock lock1 = new ReentrantLock(true);
6     private Lock lock2 = new ReentrantLock(true);
7     public static void main(String[] args) {
8         DeadLock deadLock = new DeadLock();
9         new Thread (deadLock::operation1, "T1").start();
10        new Thread (deadLock::operation2, "T2").start();
11    }
12
13    public void operation1() {
14        lock1.lock();
15        System.out.println("lock1 success, waiting to lock2");
16        try {
17            Thread.sleep(100);
18        } catch (InterruptedException e) {
19            throw new RuntimeException(e);
20        }
21
22        lock2.lock();
23        System.out.println("lock2 success");
24        System.out.println("Executing first operation");
25        lock2.unlock();
26        lock1.unlock();
27    }
28
29    public void operation2() {
30        lock2.lock();
31        System.out.println("lock2 success, waiting to lock1");
32        try {
33            Thread.sleep(100);
34        } catch (InterruptedException e) {
35            throw new RuntimeException(e);
36        }
37
38        lock1.lock();
39        System.out.println("lock1 success");
40        System.out.println("Executing second operation");
41        lock1.unlock();
42        lock2.unlock();
43    }
44 }
```

```
1 import java.util.concurrent.TimeUnit;|
2 import java.util.concurrent.locks.Lock;
3 import java.util.concurrent.locks.ReentrantLock;
4
5 public class LiveLock {
6     private Lock lock1 = new ReentrantLock(true);
7     private Lock lock2 = new ReentrantLock(true);
8
9     Run | Debug
10    public static void main(String[] args) {
11        LiveLock liveLock = new LiveLock();
12        new Thread (liveLock::operation1, "T1").start();
13        new Thread (liveLock::operation2, "T2").start();
14    }
15
16    public void operation1() {
17        while (true) {
18            try {
19                lock1.tryLock(1000, TimeUnit.MILLISECONDS);
20            } catch (InterruptedException e) {
21                throw new RuntimeException(e);
22            }
23            System.out.println("lock1 success, try to lock2");
24
25            try {
26                Thread.sleep(1000);
27            } catch (InterruptedException e) {
28                throw new RuntimeException(e);
29            }
30
31            if (lock2.tryLock()) {
32                System.out.println("lock2 success");
33            } else {
34                System.out.println("lock2 failed");
35                lock1.unlock();
36                continue;
37            }
38
39            System.out.println("Executing first operation");
40            break;
41        }
42        lock2.unlock();
43        lock1.unlock();
44    }
```

```
45     public void operation2() {
46         while (true) {
47             try {
48                 lock2.tryLock(1000, TimeUnit.MILLISECONDS);
49             } catch (InterruptedException e) {
50                 throw new RuntimeException(e);
51             }
52             System.out.println("lock2 success, try to lock1");
53
54             try {
55                 Thread.sleep(1000);
56             } catch (InterruptedException e) {
57                 throw new RuntimeException(e);
58             }
59
60             if (lock1.tryLock()) {
61                 System.out.println("lock1 success");
62             } else {
63                 System.out.println("lock1 failed");
64                 lock2.unlock();
65                 continue;
66             }
67
68             System.out.println("Executing second operation");
69             break;
70         }
71         lock1.unlock();
72         lock2.unlock();
73     }
74 }
```

```
1 public class OneAndTwo {
2     private static final Object lock = new Object();
3     /*true - ход потока 1, false - ход потока 2 */
4     private static boolean turn = true;
5     public static void main(String[] args) {
6         Thread thread1 = new Thread(() -> {
7             while (true) {
8                 synchronized (lock) {
9                     while (!turn) {
10                         try {
11                             lock.wait();
12                         } catch (InterruptedException e) {
13                             Thread.currentThread().interrupt();
14                             return;
15                         }
16                     }
17                     System.out.println("1");
18                     turn = false;
19                     lock.notify();
20                 }
21             }
22         });
23
24         Thread thread2 = new Thread(() -> {
25             while (true) {
26                 synchronized (lock) {
27                     while (turn) {
28                         try {
29                             lock.wait();
30                         } catch (InterruptedException e) {
31                             Thread.currentThread().interrupt();
32                             return;
33                         }
34                     }
35                     System.out.println("2");
36                     turn = true;
37                     lock.notify();
38                 }
39             }
40         });
41
42         thread1.start();
43         thread2.start();
44     }
45 }
```

